

01 · SYSTEM CONTEXT

THE GOLDEN RULE – “The harness is deterministic; the work is creative.” Python owns the loop, every stop decision, every model choice and every file write. LLMs are pure functions from a whitelisted prompt to JSON.

A cognitive software factory that searches for the best **context-health intervention MLP**. Four isolated LLM agents (Meta, Worker, Judge, Doc) propose, build, score and document experiments; a deterministic Python harness (`run_factory.py`) controls the loop; a pure-Python **simulator** trains and evaluates every candidate. Work proceeds in **synchronous parallel waves**: threads only compute — every shared-state mutation happens on the main thread, at a barrier, in iteration order.

4 LLM agents + 1 deterministic simulator

3 models · 4 ladder levels · 6 routing rules

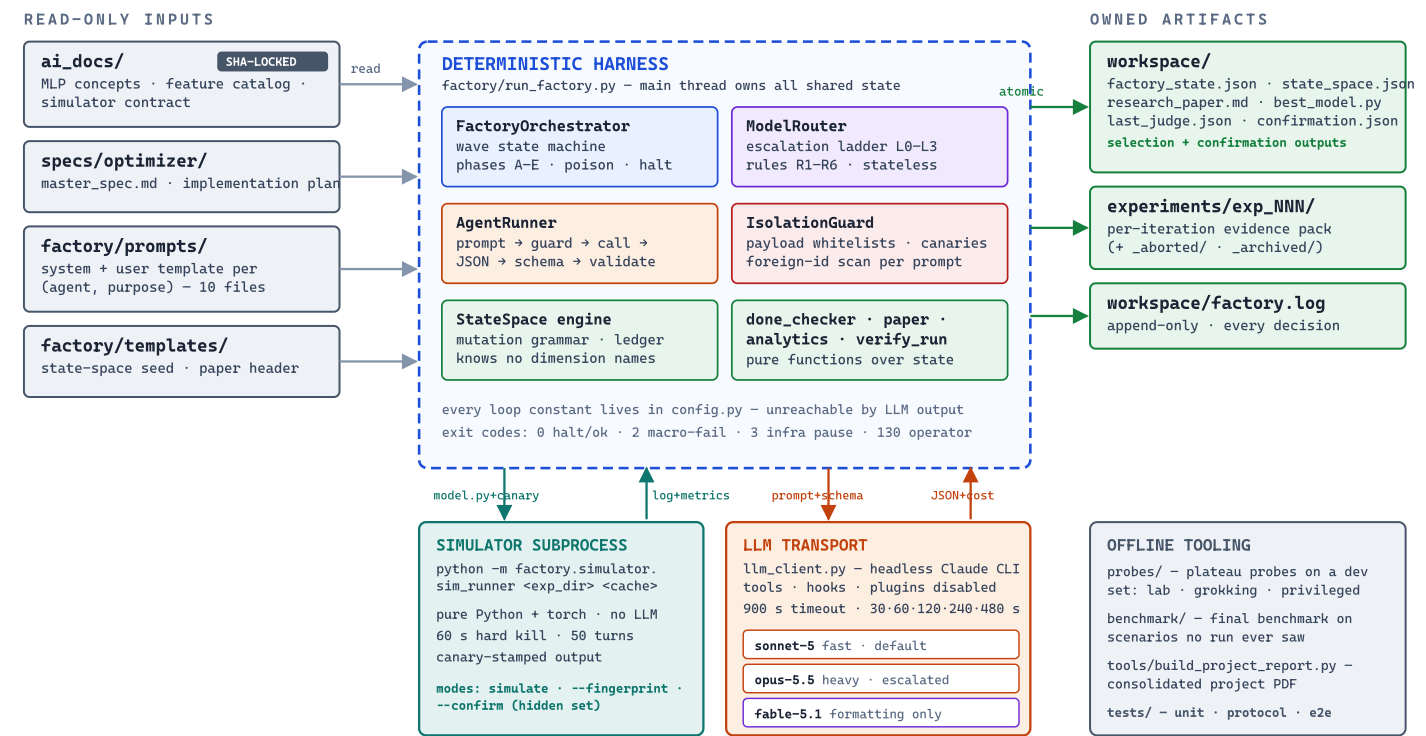
50 turns · 60 s sim budget · judge anchored ±3

3 attempts → poison · stagnation window 10

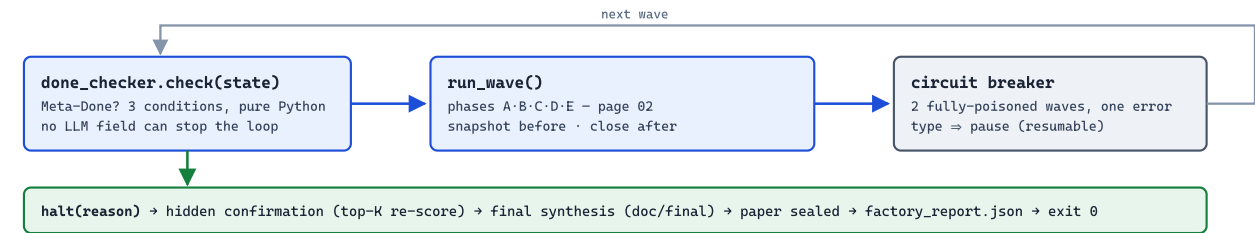
waves of 2 (parallel ≤ 8) · refills ≤ 2/wave

LLM call timeout 900 s · backoff 30·60·120·240·480 s

CONTEXT MAP – inputs · harness · transport · owned artifacts



THE LOOP AT 10,000 FT – stop decisions are Python-only



DIRECTORY CONTRACT – deliverables only inside designated roots

ROOT	MODE	CONTENTS
ai_docs/	read-only	initial MLP concepts, feature catalog, simulator contract – checksum-verified at preflight and halt
specs/optimizer/	docs	master spec, implementation plan, consultations, probe/benchmark results, project report
factory/	code	the deterministic harness: orchestrator, agents, router, isolation, simulator, prompts, tests
workspace/	owned	living state: factory_state.json, state_space.json, research_paper.md, best models, logs
experiments/	owned	exp_000 ... exp_NNN evidence packs; _aborted/ (mid-wave interrupts) and _archived/ (fresh restarts)

02 · WAVE EXECUTION PIPELINE

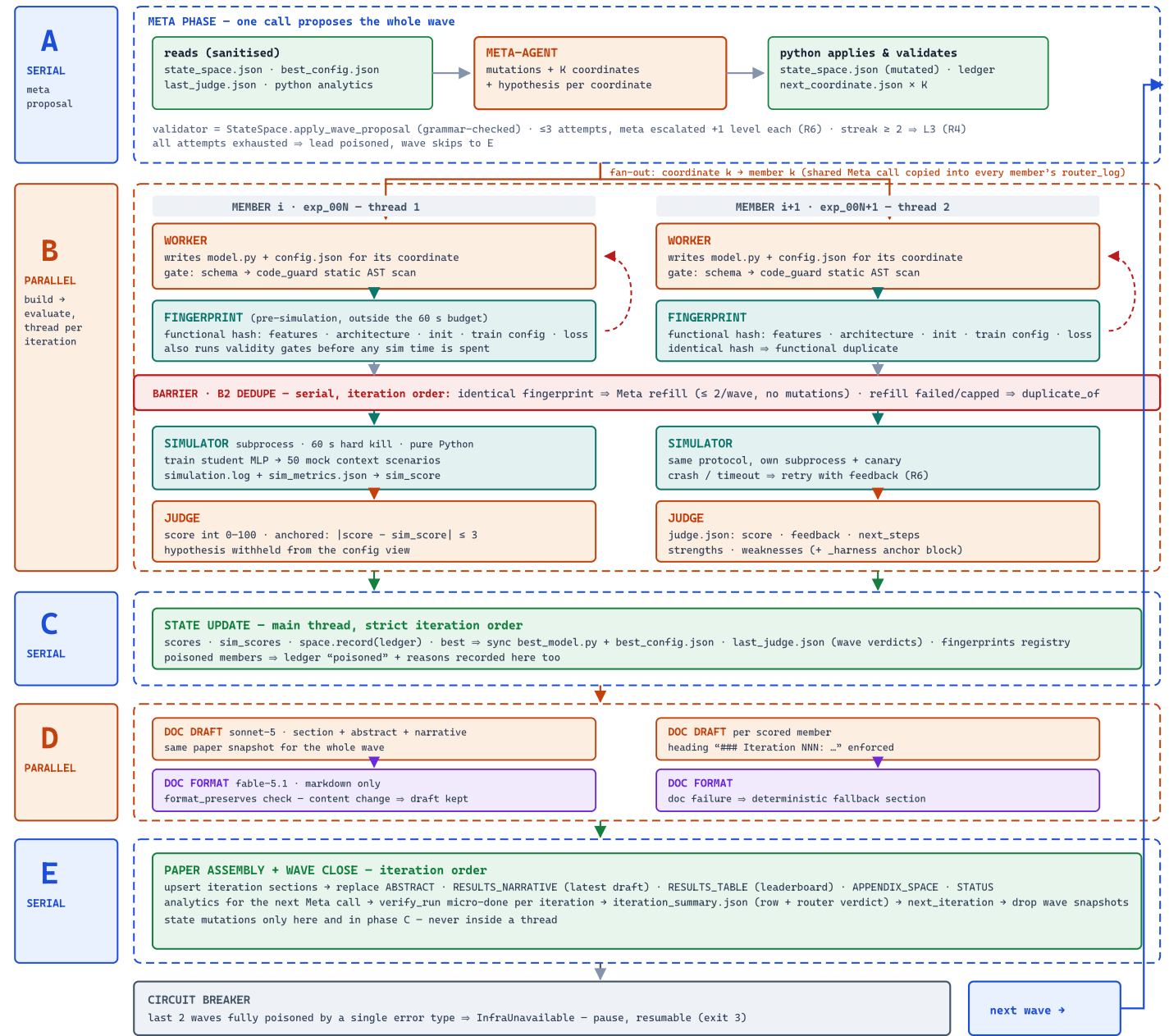
One **wave** = K iterations (default K=2, ≤8). Threads only compute; **all shared-state mutations happen on the main thread at barriers, in iteration order** — the outcome never depends on which thread finishes first. A snapshot of `state_space.json` + `factory_state.json` is taken before every wave; an interrupted wave is archived to `experiments/_aborted/` and restored on resume.

SERIAL - main thread

PARALLEL - one thread per iteration, compute-only

BARRIER - join in iteration order

retry loop per member: ≤3 attempts → poison



Memoisation: each member caches stage results (memo); a retry redoes only failed stages — except simulator/worker-code failures, which redo Worker + Fingerprint with the error fed back as `retry_error`. **Poisoned** members skip D's LLM calls and still get a deterministic paper section, so the research record is complete.

03 · AGENTS & ISOLATION

Agents are **pure functions from a whitelisted prompt to JSON**. They have no tools; the only data they ever see is what Python assembles into the prompt. Every call runs the same AgentRunner pipeline: render template → isolation check → routed LLM call → JSON extraction → schema validation → semantic validator, with **one in-place retry** (rejection feedback appended, effort forced high — rule R2).

A · META-AGENT – “The Explorer”

sonnet-5 · std → opus-5.5/high on streak

READS — state_space.json · best_config.json · last_judge.json (all sanitised; analytics are Python-computed aggregates)

WRITES — state-space mutations + K coordinates + hypotheses, applied by Python via StateSpace.apply_wave_proposal

GATES — JSON schema → grammar: ops add_dimension · add_values · prune_values · prune_dimension; ≤6 mutations/iter; ≤32 active dims; categorical ≤16 values; description 10–800 chars; ledger/wave hash uniqueness

BLIND SPOT — never sees past experiment logs; 2 consecutive invalid proposals ⇒ escalated to L3 (R4)

B · WORKER – “The Builder”

sonnet-5 · std (L2 if last score < 60)

READS — next_coordinate.json · dimension definitions · full ai_docs/* corpus · retry_error after a failed attempt

WRITES — model.py (PyTorch) + config.json for exactly its coordinate

GATES — schema → code_guard static AST scan **before any execution** → fingerprint-stage validity gates; simulator crashes return as **retry_error** feedback

BLIND SPOT — never sees past experiment logs or other wave members’ code

D · JUDGE – “The Evaluator”

sonnet-5 · std (L2 if last score < 60)

READS — simulation.log · config.json **with the hypothesis withheld** so it cannot bias the score

WRITES — judge.json : score (int 0–100) · feedback · next_steps · strengths · weaknesses

GATES — schema → **anchor**: |score – sim_score| ≤ 3 else JudgeAnchorError (micro-fail → escalate); the rubric can only move the deterministic sim_score within tolerance

BLIND SPOT — never sees past experiment logs; score must stay near the deterministic metric

E · DOC – “The Researcher”

sonnet-5 draft · fable-5.1 format only

READS — config.json · judge.json · sim summary · paper snapshot (identical for the whole wave); final synthesis: paper + factory summary

WRITES — iteration section (“### Iteration NNN: <title>”) + refreshed abstract + results narrative

GATES — heading pattern + no h1/h2 → format pass must be **content-preserving** (format_preserves) else the draft is kept → total doc failure ⇒ harness writes a deterministic fallback section

ONLY FABLE CONSUMER — fable-5.1 exists solely for its markdown formatting pass (R5)

C · SIMULATOR – “The Ground Truth”

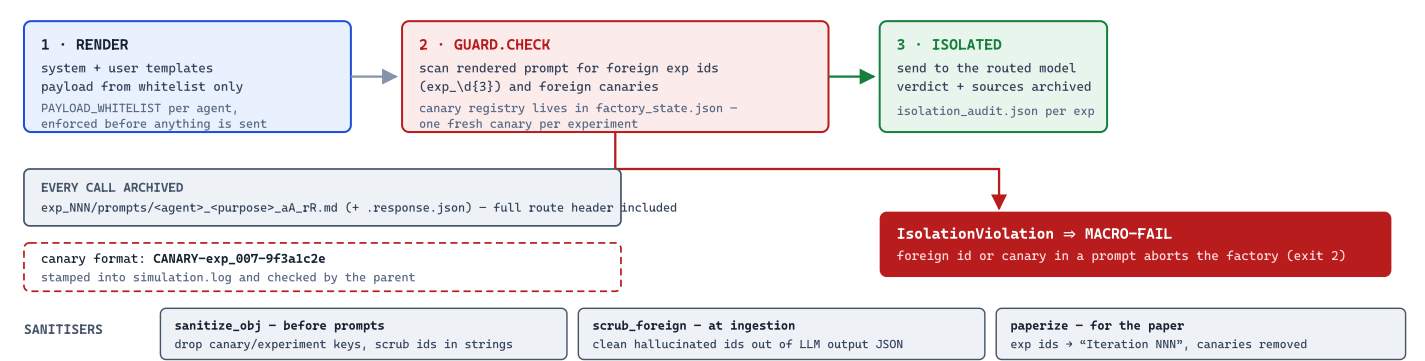
no LLM – RoutingViolationError if any model is routed to it

WHAT — deterministic Python subprocess: trains the Worker’s model.py, runs 50 mock context scenarios, writes canary-stamped log + metrics.

BUDGET — 60 s hard kill; identical env for every experiment (PYTHONHASHSEED=0, CUDA off, OMP threads pinned).

ROLE — produces sim_score , the number the Judge is anchored to and the Meta reasons over.

ISOLATION GUARD – scanned on every prompt, right before the LLM call

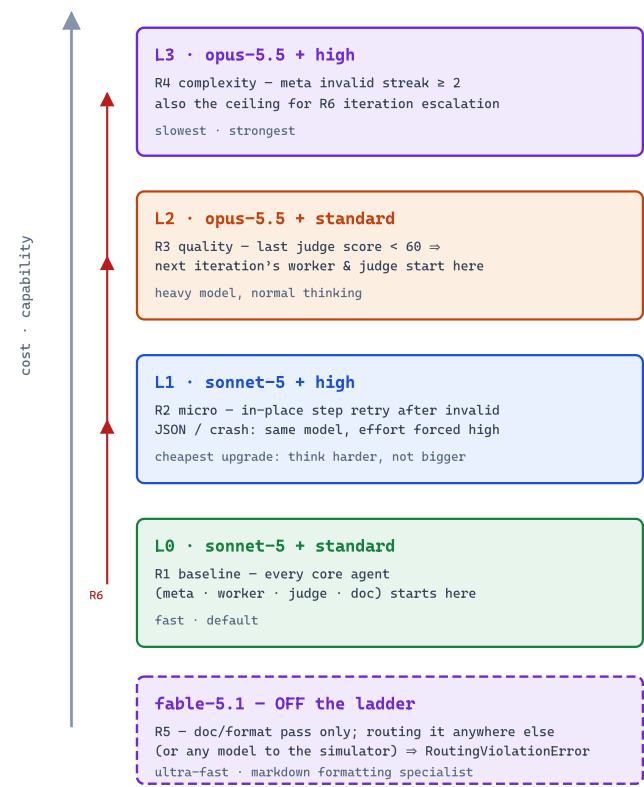


Why it matters: the Meta must reason over the *space*, not over other experiments’ logs; the Judge must score this run, not a remembered one. Isolation is what makes scores comparable across iterations — and a breach is catastrophic, so it aborts rather than degrades.

04 · MODEL & EFFORT ROUTER

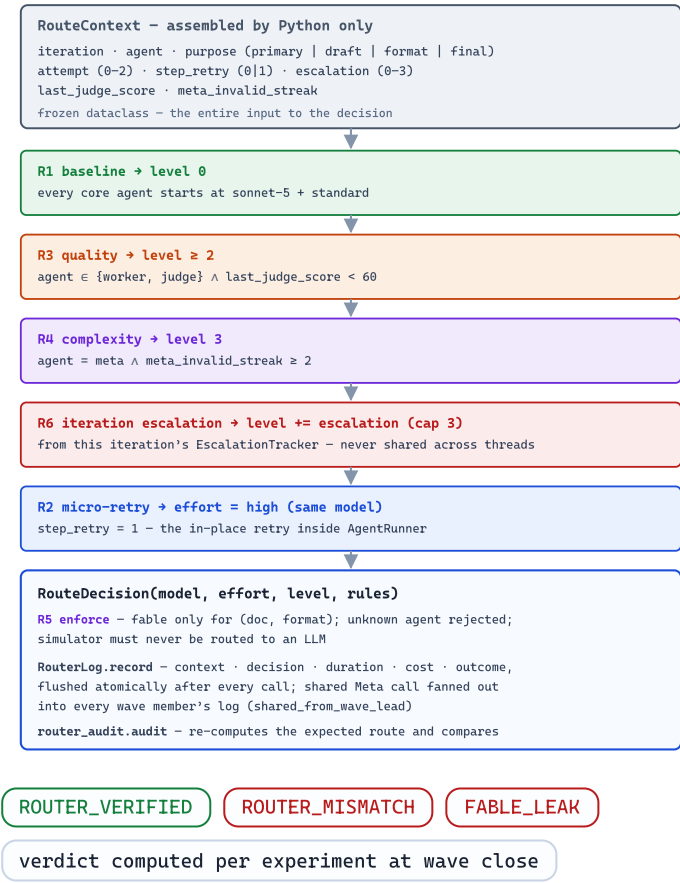
The LLM never picks a model. **Python assigns one from failure history**, using a 4-level escalation ladder and six hardcoded rules. The router is **stateless** — `route()` is a pure function of a `RouteContext`; all mutable escalation state lives in one `EscalationTracker` per iteration, so parallel iterations can never change each other's routing. Every decision is logged to `router_log.json` and independently re-audited.

THE ESCALATION LADDER



R6: each iteration-level failure bumps the failing agent one level (cap L3) — tracked per iteration by `EscalationTracker`.

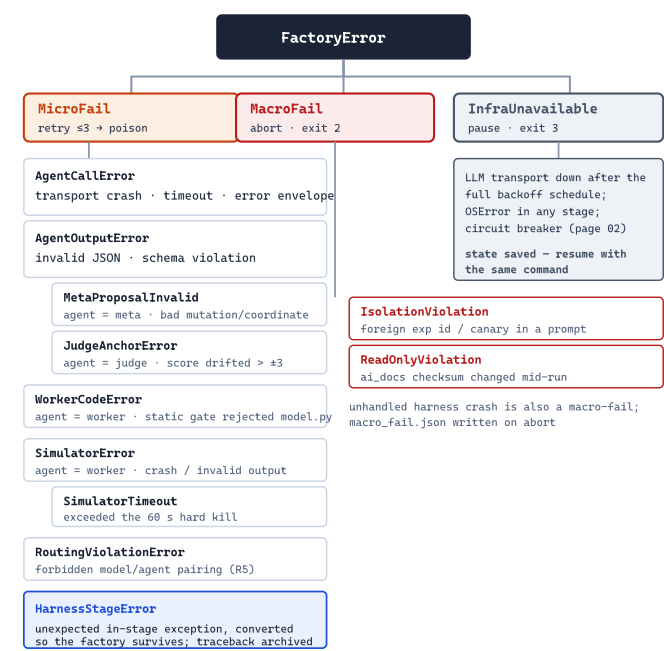
`route()` DECISION PIPELINE – rules applied in order



05 · FAILURE SEMANTICS & STOP CONDITIONS

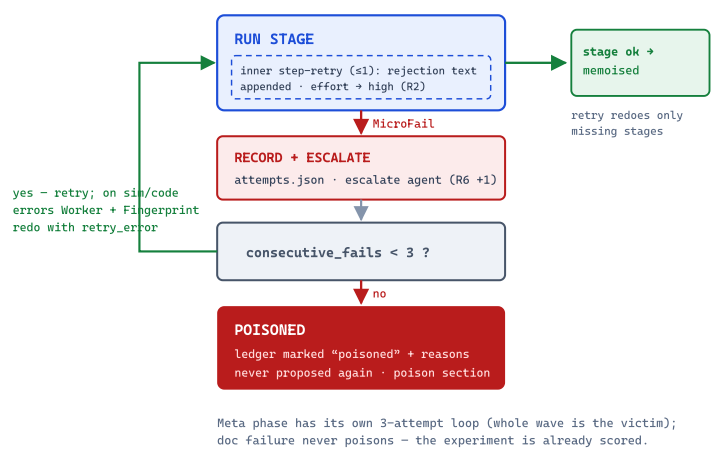
Three failure severities, three different fates. **Micro-fails** are absorbed by the iteration retry loop (escalate, retry ≤ 3 , then poison the coordinate). **Macro-fails** abort the factory instantly. **InfraUnavailable** pauses with fully resumable state. An unexpected exception inside a stage is converted to `HarnessStageError` (a micro-fail) — the factory survives its own bugs.

EXCEPTION HIERARCHY



micro-fail causes are fed back to the agents: Worker sees `retry_error`; the in-place step retry sees the harness rejection text. Failure history drives routing (R2/R6) – the same failure is never retried at the same level without escalation.

ITERATION RETRY STATE MACHINE – per member, per attempt



META-DONE CONDITIONS – checked by Python before any agent runs

#	CONDITION	CONSTANTS
1	state-space stagnation – no dimension added in the window	STAGNATION_WINDOW = 10
2	score plateau – best of the window $\leq$ earlier best + margin	MIN_IMPROVEMENT = 2.0
3	hard limit reached	--max-iterations (default 50)

EXIT CODES

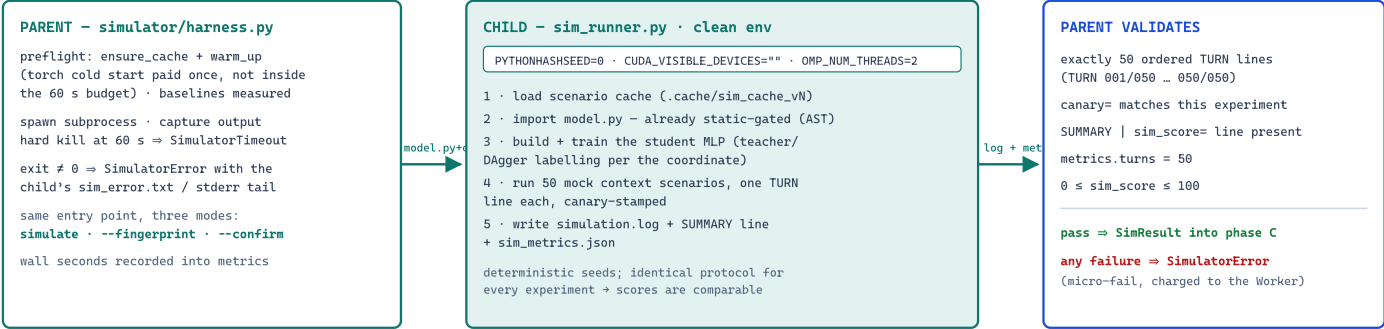
- 0 – Meta-Done halt / already halted
- 2 – macro-fail abort
- 3 – infra pause · resumable
- 130 – operator interrupt · resumable

A halt is not the end of the pipeline: **hidden confirmation** and the **final synthesis** run after it — see page 06. If the factory is rerun after halting, it exits immediately (`state["halted"]`); `--fresh` archives the previous run and reseeds.

06 · SIMULATOR & EVALUATION INTEGRITY

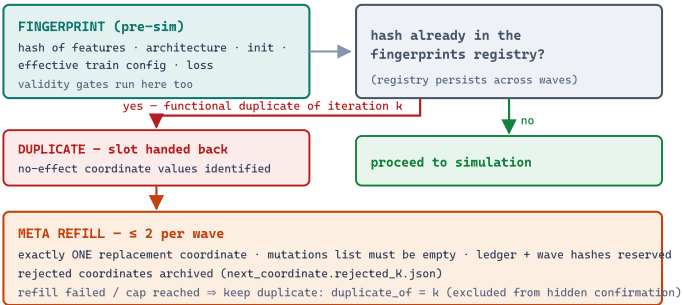
The simulator is the factory's ground truth: a **pure-Python subprocess** that trains the Worker's model on synthetic context-health scenarios and scores it. Scores stay honest through four mechanisms: **canary-stamped output**, a **functional fingerprint** that detects duplicate models, a **Judge anchored ± 3** to the deterministic `sim_score`, and a **hidden confirmation set** that breaks the evaluation set's winner's curse.

SIMULATION PROTOCOL – parent spawns, child computes, parent validates

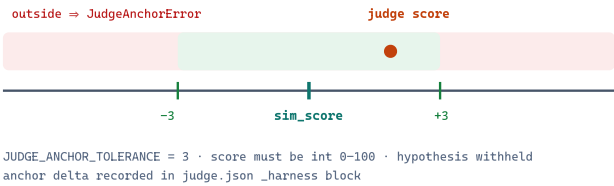


metrics – `sim_score` · `mean_return` · `success_rate` · `overflow_rate` · `oracle_agreement` · `action_entropy` · `action_distribution` · `train_acc` / `val_acc` · `features` · `params` · `wall_seconds`

FINGERPRINT → DEDUPE → REFILL



JUDGE ANCHOR – rubric within tolerance of ground truth



CALIBRATION BASELINES – measured at preflight

noop

heuristic

oracle

teacher_v2

clairvoyant

mean return per policy → `state["baselines"]`

SELECTION vs CONFIRMATION – beating the winner's curse

- Selection (spec rule):** `best_model.py` = highest judge score; ties broken by `sim_score`. Synced at phase C of every wave.
- Confirmation (harness, at halt):** re-score candidates on **200 hidden scenarios (seeds 95000+)** never used for selection.
- Candidates:** every iteration `sim`-tied within $2 \times$ the typical paired SE of the best (cap `CONFIRM_TOP_K` = 6; functional duplicates excluded).
- Statistics:** paired per-turn return differences vs the selected best; mean Δ and its SE reported per candidate.
- Outputs:** `confirmation.json` · `best_model_confirmed.py` · CONFIRMATION block in the paper · factory summary names *selected* vs *confirmed* best.
- Also at halt:** final synthesis (doc/final over paper + summary) → `factory_report.json` (verify_run macro/micro audit) → `ai_docs` re-checksumed.

07 · MODULE MAP & ARTIFACT INVENTORY

Every module in `factory/` and what it owns, plus the exact artifacts an experiment and a run leave behind. If it isn't written here, the factory didn't produce it — `verify_run` audits the definitions of done and fail independently of the orchestrator.

factory/ MODULES

MODULE	OWNS
<code>run_factory.py</code>	FactoryOrchestrator – wave state machine A-E · retry/poison · dedupe/refill · snapshots + resume · circuit breaker · halt + confirmation + final synthesis
<code>config.py</code>	every path + loop constant; unreachable by LLM output
<code>errors.py</code>	FactoryError hierarchy: MicroFail / MacroFail / InfraUnavailable
<code>done_checker.py</code>	the 3 Meta-Done conditions – pure functions over Python state
<code>state_store.py</code>	atomic JSON persistence (tmp + replace) for Python-owned state
<code>agents.py</code>	AgentRunner: render → guard → routed call → extract → schema → validate · one in-place retry · prompt archiving
<code>schemas.py</code>	JSON Schema per (agent, purpose) – meta/worker/judge/doc draft/format/final
<code>router.py</code>	ladder L0-L3 · rules R1-R6 · EscalationTracker · RouterLog + verdicts
<code>router_audit.py</code>	re-computes the expected route per decision; mismatch detection
<code>llm_client.py</code>	headless Claude CLI transport – tools/hooks off · 900 s timeout · fixed backoff · cost tracking
<code>isolation.py</code>	payload whitelists · canaries · foreign- id scan · sanitize_obj / scrub_foreign / paperize
<code>code_guard.py</code>	static AST gate for Worker model.py – before the code ever executes
<code>state_space.py</code>	mutation grammar engine (types, ops, domain checks, ledger, hashes) – knows no dimension names
<code>analytics.py</code>	Python aggregates for the Meta: iteration rows · per-dimension marginals · paired SE
<code>paper.py</code>	deterministic paper assembly: block replacement · leaderboard · space appendix · format_preserves
<code>verify_run.py</code>	independent audit of Done/Fail → factory_report.json + per-iteration micro-done
<code>init_workspace.py</code>	seed state space + paper header · ai_docs checksum · --fresh archiving
<code>export_policy.py</code>	export a trained intervention MLP → workspace/deploy package
<code>simulator/harness.py</code> · <code>sim_runner.py</code>	parent spawns child subprocess (60 s kill, validates log + metrics) · child trains + scores 50 turns; fingerprint + confirmation modes
<code>simulator/dataset.py</code> · <code>env</code> · <code>labeler</code> · <code>reference_policy</code>	scenario cache · deterministic mock env · teacher labelling (own subprocess) · harness-owned reference student
<code>probes/</code> · <code>benchmark/</code> · <code>tools/</code>	offline: plateau/grokking/privileged probes · unseen-scenario benchmark · project PDF builder
<code>tests/</code>	unit (router, audit, done, isolation, space, gate) · simulator protocol · e2e with scripted mock LLM

experiments/exp_NNN/ – EVIDENCE PACK

<code>exp_007/</code>	
└ next_coordinate.json	# Meta coordinate + hypothesis
└ (+ .rejected_K.json)	# after duplicate refills
└ config.json	# coordinate · dims · sha(model.py)
└ model.py	# Worker code – run by simulator
└ worker.json	# design notes · attempt
└ fingerprint.json	# functional hash + seconds
└ simulation.log	# 50 canary-stamped TURNS + SUMMARY
└ sim_metrics.json	# deterministic metrics (sim_score...)
└ judge.json	# score · feedback · anchor block
└ doc.json	# draft · format status
└ router_log.json	# routing decisions + verdicts
└ isolation_audit.json	# guard verdict per call
└ prompts/	# exact prompts + responses
└ attempts.json	# failure ledger (retry / poison)
└ meta.json	# wave-lead only: full Meta output
└ iteration_summary.json	# row + micro-done checks

workspace/ – LIVING STATE

<code>workspace/</code>	
└ factory_state.json	# next_iteration · scores · best
└	# canaries · fingerprints · waves
└ state_space.json	# dimensions · ledger · analytics
└ research_paper.md	# Living paper – Python blocks
└ best_model.py	# selected best (top judge score)
└ best_config.json	# its coordinate + metric digest
└ best_model_confirmed.py	# hidden-set winner
└ + confirmation.json	# its scores (200 hidden scenarios)
└ last_judge.json	# wave verdicts → next Meta input
└ factory.log	# append-only run log
└ factory_report.json	# verify_run audit at halt
└ .snapshots/	# per-wave snapshots → resume
└ final_synthesis/	# closing doc call evidence
└ deploy/	# exported policy package

Reading order for a run: `factory.log` → `factory_state.json` (scores, best) → `research_paper.md` (narrative) → `factory_report.json` (audit) → individual `exp_NNN` packs for evidence.